

拾遗队列 • task 612036a4

Agent Harness 工程

—— 模型不值钱，挽具才值钱

一条"Anthropic \$250K-\$750K 聘请会造 harness 的工程师"的病毒推文，牵出 27 万 star 的开源项目 ECC (Everything Claude Code) 和当下最热的方法论：Harness Engineering

样本仓库: github.com/affaan-m/ECC (271,019 ★, HEAD ef648e0) | 实验: 30 行迷你 harness 真实驱动 claude -p (haiku) | 产出: 2026-10-03

钩子

"\$250K-\$750K 请人造挽具"
招聘八卦式话术，全程不点名

查证

指向真项目 [affaan-m/ECC](https://github.com/affaan-m/ECC)
黑客松冠军 Affaan Mustafa 开源

真货

Harness Engineering 方法论
问题不在模型智能，在环境

疑问即目录 • 六个问题带你从吃瓜到上手

Q1

这条推文到底说了什么？"this engineer"究竟是谁？

推文全文：Anthropic 以 **\$250K-\$750K 年薪**聘请了"这位工程师"，因为他会为多智能体系统构建 **harness**，并附一段 12 分钟工作坊视频，五步教你从零搭一个：**Claude Agent SDK** → **脑手分离** → **服务端+全量日志** → **失败廉价可恢复** → **日志变记忆/技能**，还说"Anthropic 把这叫 dreaming"。

查证过程（逐层排除）：推文**始终不点名**。顺藤摸瓜发现同款话术在 LinkedIn 已病毒式流传（GenAI Works 等营销号），指向 **Affaan Mustafa** (github.com/affaan-m)：Anthropic x Forum Ventures 黑客松冠军（8 小时用 Claude Code 做出 zenith.chat），随后开源了他的全套配置 **ECC — GitHub 实测 271,019 stars**，自称"The Agent Harness Operating System"。其 LinkedIn 显示 2026-06 起与 Anthropic 有关联（Community Ambassador）。

判断

"\$250K-\$750K"大概率是把 Anthropic 公开职位薪资带和黑客松冠军的故事**揉在一起的 engagement bait**；但钩子底下藏着**真项目 + 真方法论**（推文附的长文《Harness Engineering》主体是技术内容），所以判定为**学习类**——吃瓜是入口，挽具是正餐。

Q2 Harness 是什么？跟模型、prompt 是什么关系？

Harness (挽具)：模型之外的一切运行环境。一句话定义——**模型是推理引擎，harness 决定它能看什么、能碰什么、什么能跨会话存活、什么算证据、什么时候必须停**。Dario Amodei 解释 Claude Code 诞生时原话："Of course, you need an interface, you need a harness to use them"。



图1 · 模型（红）泡在 harness（绿框）提供的六个组件里

金句对照：**Prompt engineering 改进指令本身，Harness engineering 改进指令被执行的条件**。OpenAI 在《Harness engineering: leveraging Codex in an agent-first world》里也承认：agent 进展慢是因为“**环境欠规格**” (underspecified)，不是模型缺能力——解法不是让 agent “再努力试试”，而是补上缺失的能力并让它**可读 (legible) 且可强制 (enforceable)**。

Q3 为什么说“同一个模型，换个挽具就是完全不同的 agent”？

同一个模型：放进**聊天框**，它只能答题；放进带**终端、测试、浏览器工具、项目记忆、隔离 worktree、审查循环**的仓库里，它能交付软件。**权重一个字节都没变，变的是挽具**。这解释了为什么“换了模型还是老样子”——你只换了马，没换车。

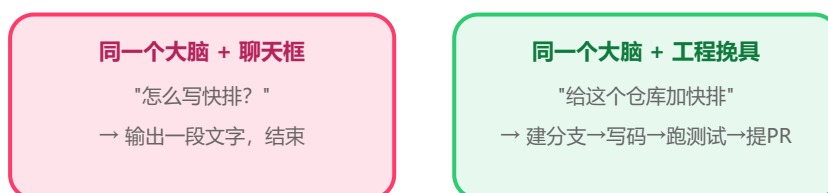


图2 · 权重没变，harness 变了，产出天差地别

Q4 生产级 harness 到底要干哪些活？（七职责总纲）

#	职责	一句话解释	不做会怎样
1	请求一契约	动手前把需求固化成有边界的对象	agent 悄悄换了活儿还宣告成功
2	给一张地图	小根指南指路，详情贴在代码旁按需加载	巨型手册吃光上下文
3	最小权限工具	正确环境暴露正确工具，写权限只给该给的目录	误删文件 / 全盘暴露
4	验证与证据	说 DONE 不算数，产出物必须真实存在	假成功静默累积
5	上下文工程	预算、压缩、模型分级路由	账单爆炸 / 长会话失忆
6	失败廉价	重试死沙箱、从日志重放，不从头再来	一次崩溃毁掉全部进度
7	停止条件	明确什么算完成、什么必须停	死循环烧钱

口诀 agent 反复失败时，**别再改 prompt 里的形容词，去检查模型周围这套系统。**

Q5 "契约"和"地图"听起来玄，实际是什么？

契约：一份机器可校验的任务边界，比如"只准写 output/ 目录、产出物是 task_a.md、少于 10 字算失败"。它是**防 agent 私改需求**的合同——没有它，agent 可以完成另一件事还理直气壮。和机器人里的 [[策略观测向量与动作契约]] 同思想：**接口先钉死，两边才不会各说各话。**

地图：一个小小的根指南（CLAUDE.md / index），只告诉 agent"什么知识放在哪"，**不把所有文档塞进上下文**。详细知识贴在它治理的代码旁边，任务需要时才加载——这正是 [[分层按需加载]]：小入口常驻，大主体待命。nihaixia skill 实测入口只占 2.71%，就是同一原理。

Q6 ECC 是怎么把这套东西做成"操作系统"的?

ECC v2.0.0 (2026-06) 的仓库就是一部 harness 教科书: **293 个 skills** (按需加载的工作流, 主表面)、**68 个 agents** (YAML frontmatter 定义 name/tools/model 的子代理)、**94 个 commands** (斜杠兼容层)、**23 组 rules** (常驻规范)、**hooks** (PreToolUse/PostToolUse/Stop 事件自动化)、**contexts** (动态系统提示注入), 并适配 Codex/OpenCode/Cursor。

ECC 组件	对应七职责	亮点
agents/ (68 个子代理)	分工+地图	python-reviewer、rust-build-resolver、harness-optimizer.....按语言/场景各司其职
skills/ (293 个工作流)	地图 (按需加载)	tdd-workflow、verification-loop、strategic-compact.....
hooks/ (事件自动化)	验证+观察	continuous-learning-v2.1 用 PreToolUse/PostToolUse 100% 可靠观察会话
instinct 本能体系	记忆	后台小模型把会话提炼成 原子本能 (置信度 0.3~0.9), 项目作用域隔离, 攒够了聚类进化成 skill—— harness 越用越聪明
Token 优化配置	上下文工程	sonnet 打底 opus 攻坚 (~60% 降本)、MAX_THINKING_TOKENS 10000、50% 早压缩、MCP<10 个 ([[工具目录膨胀]])

清醒剂 个人日常**不必全装 ECC**——装满会吃光上下文。挑 2~3 个职责自己实现 mini 版 (见实验), 比整套安装更能学会 harness 思维。

动手实验: 30 行核心逻辑, 搭一个会成长的迷你 Harness

材料在 `agent-harness\exercise\`: `mini_harness.py` (harness 本体) + `contract.json` (契约)。驱动无头 `claude -p` (haiku)。以下每一步都在 Windows 11 真实执行, 输出为实拍。

STEP 1 • 写契约: 把"可以做什么"钉死在 JSON 里

操作: `contract.json` 定义 scope ("只允许写 exercise/output/ 下的 markdown") + 任务表 (task_a/task_b)。
目的: 职责①契约——模型还没出场, 边界先立好。检测: 每次运行 start 事件都把 scope 记进日志。

STEP 2 • 跑第一个任务: 契约 → 调用 → JSONL 日志

操作: `python mini_harness.py run task_a`
目的: 职责⑥日志——每一步 (start/success/fail/learn) 追加进 `logs/run_log.jsonl`, 带时间戳、耗时、花费、轮数。

✅ 预期/实测: `[OK] task_a attempt=1 20.2s cost=$0.0413 turns=2`, 产出 `output/task_a.md` (模型写出了"挽具把模型的决策变成真实动作"的中文说明)。

STEP 3 • 验证门：说 DONE 不算数

操作：harness 在模型回复后执行 `verify()`——产出文件不存在或 <10 字节，直接判 fail 进重试。

目的：职责④验证与证据。实拍名场面：第一版 harness 里模型回复“写入完成”，但 `permission_denials` 审计显示 Write 被无头模式拒绝（模型连完整内容都写好了！）——验证门把这次“假成功”拦下，重试 3 次全部记录在案。

✓ 预期：日志出现 `"error": "产出物缺失或过小"` ×3，随后修复权限（`--permission-mode acceptEdits`）后 `attempt=1` 即 success。

STEP 4 • 失败廉价 + 换模型不换挽具

操作：故意制造失败（多行 prompt 经 `cmd.exe` 断行）观察重试；改 `MODEL = "haiku"` 一行即可切换底层模型。

目的：职责⑥失败廉价（重试 + 审计定位根因）+ 证明 harness 与模型解耦。

✓ 预期/实测：日志连续 3 条 fail（每条带 attempt 和错误摘要），harness 不崩、可定位、改一行配置即恢复。日志不是装饰，是排障的第一现场。

STEP 5 • learn 命令：让 harness 醒来时更聪明

操作：`python mini_harness.py learn task_a` “本项目所有产出的 markdown，标题行必须以 `[HARNESS]` 前缀开头”，然后 `python mini_harness.py run task_b`——harness 自动把 `memory.md` 注入新任务的 prompt，任务文本里只字未提该约定。

目的：职责④记忆——昨天日志/教训固化，下次自动生效（ECC instinct 的最小雏形）。

✓ 预期/实测：`[OK] task_b attempt=1 46.7s cost=$0.0542`，打开 `output/task_b.md`——标题第一个词就是 `[HARNESS]`。模型没被告知，却遵守了它没见过的约定：这就是“harness 会成长”。

踩坑实录 实验全程踩了 4 个真实的坑，每一个恰好对应一条 harness 职责：① 多行 prompt 经 `cmd.exe` 断行→调用全灭（→日志审计定位）② 模型谎报 DONE（→验证门拦截）③ 无头模式默认拒绝 Write，且 `--allowedTools` 是变长参数会吞掉后面的 prompt（→`--permission-mode acceptEdits`）④ learn 写入的记忆带尾换行，注入后 prompt 被截断（→脏记忆必须清洗）。

这个项目给我什么 • 能力清单

① 七职责检查表

agent 不听话时按表排查，不再盲目改 prompt

② 30 行迷你 harness

契约/日志/重试/验证/记忆五件套，直接抄去管自己的 AI 任务

③ 无头 claude -p 用法

`--output-format json` + permission 模式 + `permission_denials` 审计字段

④ ECC 选装思路

293 skills 按需挑：tdd-workflow / verification-loop / strategic-compact

⑤ instinct 记忆模式

教训→原子条目→置信度→固化成 skill，让任何工具越用越顺手

⑥ 营销鉴别力

不点名的薪资八卦底下可能藏真项目——看实质指向，不看话术

一句话总结：别再调教马了，去造一辆好车。模型每半年换一代，而你写的 harness——契约、日志、验证门、记忆——可以一直用下去。

Agent Harness 工程 · 拾遗队列 task 612036a4 · 2026-10-03 | 样本: affaan-m/ECC @ ef648e0 (271,019 ★) | 方法论来源: rari
《Harness Engineering》长文 (fxtwitter API 全文提取)、OpenAI 《Harness engineering: leveraging Codex in an agent-first world》 | 实验全部真实运行于 Windows 11 + claude CLI 2.1.287 (haiku, 合计花费约 \$0.14) | 知识库: vault/概念/AgentHarness智能体挽具 · 本能学习与置信度评分 · 上下文预算与战略压缩